

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

Module Machine Learning — GI2

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par : Eya Hadj Abdallah

Module : Machine Learning — GI2

Encadrante : Mme Fadoua Drira

Année universitaire : 2025–2026

Chaîne complète de traitement

Exploration → Préparation → Modélisation
→ Évaluation → Déploiement

Table des matières

1	Introduction	2
1.1	Contexte et problématique métier	2
1.2	Objectifs du projet	2
1.3	Structure du rapport	2
2	Exploration et analyse des données	2
2.1	Chargement et premières observations	2
2.2	Diagnostic de qualité des données	3
2.3	Analyse des corrélations	4
3	Préparation des données	4
3.1	Nettoyage initial	4
3.2	Feature engineering	4
3.3	Détection et correction du data leakage	4
3.3.1	Définition et impact	4
3.3.2	Colonnes fuyantes supprimées	5
3.4	Encodage des variables catégorielles	5
3.5	Split train/test et normalisation	5
4	Modélisation	5
4.1	Analyse en Composantes Principales (ACP)	5
4.1.1	Objectif et justification	6
4.1.2	Résultats	6
4.2	Clustering non supervisé — K-Means sur composantes ACP	6
4.2.1	Objectif	6
4.2.2	Choix du nombre de clusters	6
4.2.3	Profils des quatre segments	7
4.3	Classification supervisée du churn	7
4.3.1	Random Forest	7
4.3.2	XGBoost	8
4.3.3	Stacking (Random Forest + XGBoost)	9
4.4	Analyse de l'importance des features	10
4.5	Courbes ROC comparatives	11
4.6	Régression — Prédiction du montant total dépensé	12
4.6.1	Passage au XGBoost Regressor	12
5	Évaluation et synthèse des résultats	13
5.1	Comparaison globale des modèles de classification	13
5.2	Impact de la correction du data leakage	13
5.3	Recommandations métier	14
6	Déploiement — Application web Flask	14
6.1	Architecture de l'application	14
6.2	Fonctionnalités	14
7	Conclusion	15
7.1	Bilan du projet	15
7.2	Limites et perspectives	15
A	Annexe : Structure du projet	16

1 Introduction

1.1 Contexte et problématique métier

Dans un marché e-commerce de plus en plus concurrentiel, la fidélisation client représente un enjeu stratégique majeur. Acquérir un nouveau client coûte en moyenne cinq à sept fois plus cher que de retenir un client existant. C'est pourquoi l'entreprise de vente en ligne de cadeaux qui nous a confié ce projet cherche à anticiper les départs de clients — phénomène désigné par le terme *churn* — afin d'agir de manière proactive avant qu'il ne soit trop tard.

“Comment identifier, à partir des données transactionnelles et comportementales des clients, ceux qui présentent un risque élevé de churn, afin de déclencher des actions de fidélisation ciblées et opportunes ?”

Pour répondre à cette question, l'entreprise dispose d'une base de données riche : 4 372 clients décrits par 52 variables couvrant leurs habitudes d'achat (fréquence, montant, récence), leurs préférences comportementales (saison, heure, région), leur profil démographique (âge, pays) et des indicateurs de satisfaction. Cette base est volontairement imparfaite — elle contient des valeurs manquantes, des anomalies et des variables redondantes — afin de simuler les conditions réelles rencontrées en entreprise.

1.2 Objectifs du projet

Ce projet poursuit trois objectifs métier interdépendants :

1. **Segmenter la clientèle** : regrouper les clients en profils homogènes (clustering non supervisé) pour personnaliser les stratégies marketing par segment.
2. **Prédire le churn** : construire un modèle de classification supervisé capable d'identifier les clients à risque de départ avec une haute précision.
3. **Prédire la valeur client** : estimer le montant total dépensé par un client (régression) pour hiérarchiser les efforts de fidélisation selon la valeur économique.

1.3 Structure du rapport

Ce rapport suit la chaîne complète d'un projet de data science telle que définie dans le cahier des charges : exploration des données, préparation (nettoyage, feature engineering, correction du data leakage, normalisation), modélisation (ACP, K-Means, Random Forest, XGBoost, Stacking, XGBoost Regressor), évaluation comparative et déploiement Flask.

2 Exploration et analyse des données

2.1 Chargement et premières observations

Le jeu de données `retail_customers_COMPLETE_CATEGORICAL.csv` est chargé avec la bibliothèque `pandas`. Une première inspection via `df.shape`, `df.dtypes` et `df.describe()` révèle les caractéristiques générales du tableau 1.

Le tableau 1 résume les dimensions et la distribution des classes du jeu de données.

TABLE 1 – Caractéristiques générales du jeu de données

Caractéristique	Valeur
Nombre d’observations	4 372 clients
Nombre de variables totales	52 features
Variable cible	Churn (binaire : 0 = fidèle, 1 = parti)
Clients fidèles (Churn = 0)	2 929 clients — 67%
Clients churners (Churn = 1)	1 443 clients — 33%
Features numériques	34 variables
Features catégorielles	18 variables

Déséquilibre des classes

La cible **Churn** présente un ratio de 2 :1 en faveur des clients fidèles (67% vs 33%). Ce déséquilibre doit être pris en compte lors de la modélisation pour éviter que le modèle ne soit biaisé vers la classe majoritaire. Nous utiliserons **SMOTE** pour rééquilibrer les données d’entraînement.

2.2 Diagnostic de qualité des données

Une analyse exhaustive de la qualité a permis d’identifier sept catégories de problèmes, résumées dans le tableau 2.

Le tableau 2 présente les anomalies détectées et les traitements appliqués pour y remédier.

TABLE 2 – Problèmes de qualité identifiés et traitements appliqués

Type de problème	Features concernées	Ampleur	Traitement appliqué
Valeurs manquantes	Age, AvgDaysBetweenPurchase	30% pour Age	Imputation par la médiane calculée sur le train uniquement
Valeurs aberrantes	SupportTickets (val. -1, 999), SatisfactionScore (val. -1, 99)	5-8%	Remplacement par NaN puis imputation
Formats inconsistants	RegistrationDate (formats UK, ISO, US)	100% des lignes	Parsing avec <code>pd.to_datetime(dayfirst=True)</code>
Feature sans variance	NewsletterSubscribed (toujours “Yes”)	100% constante	Suppression immédiate
Données brutes	LastLoginIP (adresse IP brute)	100% des lignes	Transformation en booléen <code>IsPrivateIP</code>
Déséquilibre des classes	Churn	33% / 67%	SMOTE (<code>sampling_strategy=0.5</code>) sur le train
Multicolinéarité	MonetaryMin, MonetaryMax, MonetaryStd, etc.	$r > 0,8$	Suppression des variables redondantes (VIF)

2.3 Analyse des corrélations

Une matrice de corrélation (heatmap) a été calculée sur l'ensemble des variables numériques. Plusieurs paires présentent une corrélation très élevée, signe de redondance informationnelle : `MonetaryMin` ↔ `MonetaryStd` : $r = 0,97$; `TotalQuantity` ↔ `AvgQtyPerTrans` : $r = 0,91$; `UniqueProducts` ↔ `UniqueDesc` : $r = 0,88$. Le VIF (*Variance Inflation Factor*) a confirmé une multicolinéarité sévère ($VIF > 10$) pour plusieurs features monétaires, justifiant leur suppression.

3 Préparation des données

3.1 Nettoyage initial

Les opérations de nettoyage ont été implémentées dans `preprocessing.py` et appliquées dans l'ordre suivant :

1. **Suppression des colonnes sans valeur prédictive** : `CustomerID` (identifiant unique sans information comportementale) et `NewsletterSubscribed` (variance nulle).
2. **Correction des valeurs aberrantes** : les valeurs codées comme erreurs de saisie (-1 , 999 pour `SupportTickets`; -1 , 99 pour `SatisfactionScore`) sont remplacées par `NaN`.
3. **Normalisation de `RegistrationDate`** : parsing multi-format puis extraction de `RegYear`, `RegMonth`, `RegDay` et `RegWeekday`.
4. **Transformation de `LastLoginIP`** : conversion en booléen `IsPrivateIP` (1 si IP privée RFC 1918, 0 sinon).

3.2 Feature engineering

Trois nouvelles variables ont été construites pour capturer des ratios métier significatifs :

Dépense quotidienne moyenne :

$$\text{MonetaryPerDay} = \frac{\text{MonetaryTotal}}{\text{Recency} + 1}$$

Valeur moyenne du panier :

$$\text{AvgBasketValue} = \frac{\text{MonetaryTotal}}{\text{Frequency}}$$

Ratio ancienneté / activité récente :

$$\text{TenureRatio} = \frac{\text{Recency}}{\text{CustomerTenureDays} + 1}$$

Remarque importante

`TenureRatio` a été construite à titre exploratoire, mais elle présente une corrélation trop élevée avec `Churn` ($r = 0,86$). Elle a donc été supprimée lors de la correction du data leakage.

3.3 Détection et correction du data leakage

3.3.1 Définition et impact

Le *data leakage* survient lorsque des informations directement dérivées de la variable cible sont incluses parmi les features d'entraînement. Le modèle apprend alors à "tricher" : ses performances paraissent parfaites en validation, mais le modèle est totalement inutilisable en production.

Symptôme détecté

Lors des premiers essais, le Random Forest atteignait une accuracy de **100%** sur l'ensemble de test — résultat trop parfait pour être honnête. Après correction rigoureuse du leakage, l'accuracy est tombée à **94,5%**, valeur réaliste et scientifiquement crédible.

3.3.2 Colonnes fuyantes supprimées

Chaque colonne a été analysée selon deux critères : corrélation avec **Churn** (seuil $> 0,85$) et sémantique métier. Le tableau 3 détaille les suppressions.

Le tableau 3 recense l'ensemble des colonnes éliminées pour cause de data leakage, classées par catégorie.

TABLE 3 – Colonnes supprimées pour cause de data leakage

Catégorie	Colonnes supprimées
Construites à partir du churn	ChurnRiskCategory, CustomerType, LoyaltyLevel, SpendingCategory, RFMSegment, AccountStatus
Comportementales post-churn	ReturnRatio, NegQtyCount, ZeroPriceCount, CancelledTransactions
Fortement corrélées à Churn	CustomerTenureDays, FirstPurchase, Age
Valeurs aberrantes non corrigeables	SupportTicketsCount, SatisfactionScore
Corrélation directe ($r > 0,85$)	Recency ($r = 0,86$), TenureRatio

Un mécanisme de contrôle automatique a été intégré dans `preprocessing.py` : si une feature présente une corrélation $> 0,85$ avec **Churn** sur les données d'entraînement, le script affiche une alerte et stoppe l'exécution.

3.4 Encodage des variables catégorielles

- **Encodage ordinal** : variables à ordre naturel — `AgeCategory`, `BasketSizeCategory`, `PreferredTimeOfDay`.
- **Encodage One-Hot** : variables nominales — `FavoriteSeason`, `Region`, `WeekendPreference`, `ProductDiversity`, `Gender`.
- **Encodage One-Hot différé pour Country** : 37+ modalités, appliqué *après* le split train/test avec `handle_unknown='ignore'` pour éviter toute fuite d'information.

3.5 Split train/test et normalisation

- **Split stratifié 80/20** (`random_state=42`) : 3 497 clients en entraînement, 875 en test. La stratification préserve le ratio 67%/33% dans les deux sous-ensembles.
- **Imputation** : `SimpleImputer` (médiane) ajusté sur `X_train` uniquement, puis appliqué à `X_test`.
- **Standardisation** : `StandardScaler` ajusté sur `X_train`, appliqué à `X_test` — aucune fuite de données.

4 Modélisation**4.1 Analyse en Composantes Principales (ACP)**

4.1.1 Objectif et justification

Après toutes les étapes de préparation, le dataset comporte **87 features** (suite aux encodages One-Hot). Une telle dimension entraîne un risque de sur-apprentissage (malédiction de la dimensionnalité), un temps de calcul accru et des difficultés de visualisation. L'ACP projette ces 87 dimensions dans un espace réduit tout en conservant le maximum de variance.

4.1.2 Résultats

L'ACP est réalisée sur les données standardisées `X_train_scaled`. Comme le montre la figure 1, **56 composantes principales** suffisent à conserver **95,5% de la variance totale**. Les 31 dimensions supprimées correspondaient essentiellement à du bruit et à des redondances déjà partiellement traitées lors du preprocessing. Ces 56 composantes sont ensuite utilisées comme entrée du K-Means.

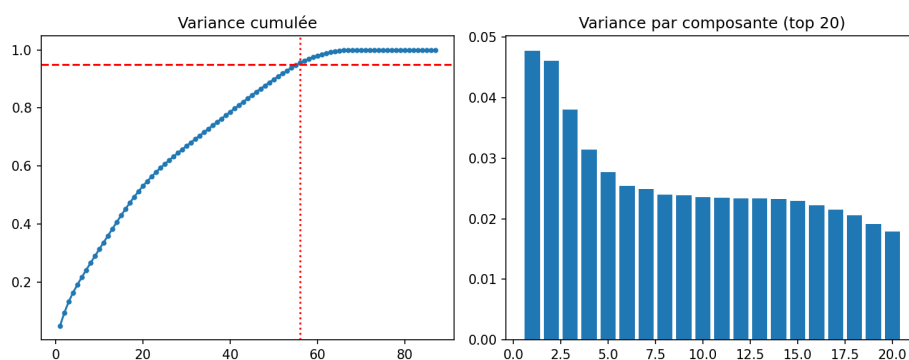


FIGURE 1 – Variance expliquée cumulée par l'ACP. La ligne verticale indique le seuil de 95,5% atteint avec 56 composantes sur 87 features initiales.

4.2 Clustering non supervisé — K-Means sur composantes ACP

4.2.1 Objectif

Le clustering K-Means vise à segmenter la clientèle en groupes homogènes sans utiliser l'étiquette `Churn`. Appliqué sur les 56 composantes ACP (et non sur les 87 features brutes), il bénéficie d'un espace moins bruité, ce qui améliore la qualité des clusters.

4.2.2 Choix du nombre de clusters

Deux critères complémentaires guident le choix de k (figure 2) :

- **Méthode du coude** : l'inertie intra-cluster décroît fortement jusqu'à $k = 4$, puis plus lentement. Le coude est clairement visible à $k = 4$.
- **Score de silhouette** : pour $k = 4$, le score est de **0,086** — une amélioration notable par rapport à 0,038 obtenu sans ACP, confirmant l'apport de la réduction dimensionnelle.

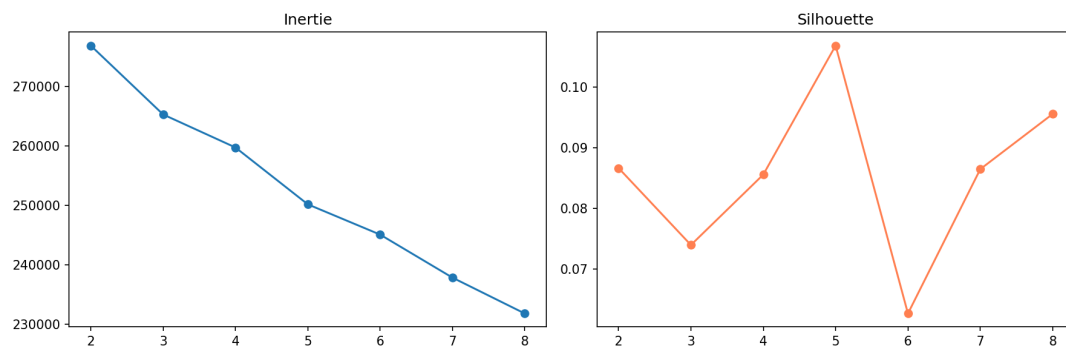


FIGURE 2 – Sélection du nombre de clusters : méthode du coude (inertie, courbe bleue) et score de silhouette (courbe orange). Le compromis optimal est $k = 4$.

4.2.3 Profils des quatre segments

Le tableau 4 décrit les quatre segments obtenus, leur taille et leurs caractéristiques comportementales.

TABLE 4 – Profils des quatre segments clients issus du K-Means ($k = 4$, sur composantes ACP)

Seg.	Nom	Taille (part)	Profil comportemental
A	Clients Premium	1 453 — 33,2%	Haute fréquence, montant élevé, faible récence. Clients les plus fidèles et rentables.
B	Clients Réguliers	884 — 20,2%	Fréquence et montant modérés. Clients stables à consolider.
C	Clients Occasionnels	928 — 21,2%	Fréquence faible, achats sporadiques. Fort potentiel de réactivation.
D	Clients Inactifs	232 — 5,3%	Très longue récence, aucune activité récente. Risque élevé de churn.

4.3 Classification supervisée du churn

Trois modèles de classification ont été entraînés pour prédire **Churn**. Chaque modèle suit le même pipeline : rééquilibrage SMOTE, optimisation des hyperparamètres par validation croisée, puis optimisation du seuil de décision.

SMOTE (*Synthetic Minority Oversampling TEchnique*) génère des exemples synthétiques de la classe minoritaire (churners) par interpolation entre voisins proches. Le paramètre `sampling_strategy=0.5` porte le ratio churners/fidèles de 1 :3 à 1 :2, sans sur-générer artificiellement.

4.3.1 Random Forest

Le Random Forest est un ensemble de T arbres de décision entraînés sur des sous-ensembles aléatoires des données (bagging) et des features (feature subsampling). La prédiction finale est obtenue par vote majoritaire : $\hat{y} = \text{mode}(h_1(x), \dots, h_T(x))$.

Les hyperparamètres ont été optimisés par **RandomizedSearchCV** (60 combinaisons, 5 plis) : `n_estimators` $\in \{600, 800, 1000, 1200\}$, `max_depth` $\in \{20, 30, 40, \text{None}\}$, `min_samples_split` $\in \{2, 3, 5\}$, `min_samples_leaf` $\in \{1, 2, 3\}$, `max_features` $\in \{\text{'sqrt'}, \text{'log2'}, 0.7\}$, `max_samples` $\in \{0.8, 0.9, \text{None}\}$. Le meilleur modèle retenu : `n_estimators=1000`, `max_depth=40`, `min_samples_split=2`, `min_samples_leaf=3`, `max_samples=0.8`, `max_features=0.7`. Le seuil de décision a été optimisé sur une plage $[0,25; 0,75]$: le meilleur seuil trouvé est **0,47** ($F1 = 0,929$), ce qui améliore légèrement l'accuracy (94,5% contre 94,4% avec le seuil 0,5).

Le tableau 5 présente le rapport de classification du Random Forest sur l'ensemble de test (seuil optimisé 0,47).

TABLE 5 – Rapport de classification — Random Forest (seuil = 0,47)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,95	0,97	0,96	584
Churner (1)	0,94	0,89	0,92	291
Accuracy globale		94,5%		

La figure 3 présente la matrice de confusion du Random Forest sur l'ensemble de test. On observe que le modèle détecte correctement la grande majorité des churners (peu de faux négatifs), ce qui est crucial pour l'objectif métier de rétention. L'AUC atteint 0,985, confirmant l'excellent pouvoir discriminant du modèle.

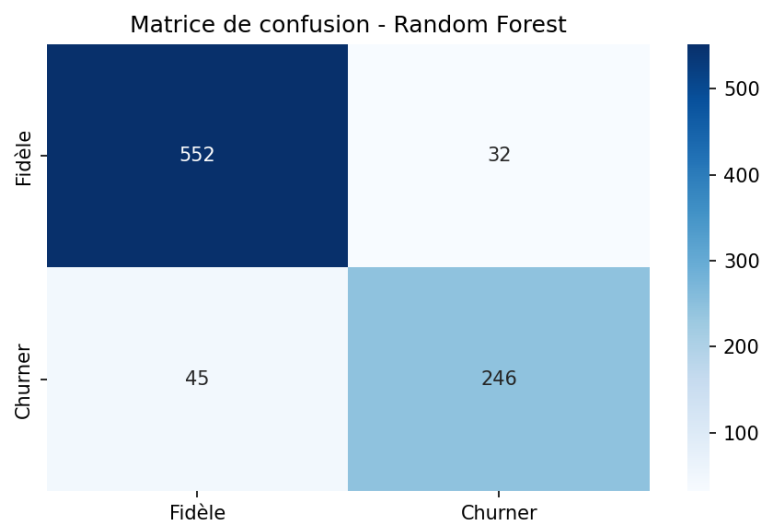


FIGURE 3 – Matrice de confusion — Random Forest (accuracy = 94,5%). Les valeurs diagonales représentent les prédictions correctes ; les valeurs hors diagonale sont les erreurs.

4.3.2 XGBoost

XGBoost (*eXtreme Gradient Boosting*) construit les arbres itérativement : chaque arbre corrige les erreurs résiduelles du précédent en minimisant une fonction de perte régularisée :

$$\mathcal{L}(\phi) = \sum_i \ell(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

où T est le nombre de feuilles, w les poids des feuilles, γ et λ des paramètres de régularisation.

Les hyperparamètres ont été optimisés par **GridSearchCV** (5 plis) : `n_estimators` ∈ {500, 700}, `max_depth` ∈ {5, 6, 7}, `learning_rate` ∈ [0,03 ; 0,08], `subsample` ∈ [0,8 ; 0,9], `colsample_bytree` ∈ [0,8 ; 0,9]. Meilleur modèle retenu : `n_estimators=700`, `max_depth=5`, `learning_rate=0,03`, `subsample=0,8`, `colsample_bytree=0,9`.

Optimisation du seuil de décision

Une recherche fine du seuil sur [0,30 ; 0,70] (pas de 0,01) a montré qu'un seuil de **0,60** maximise l'accuracy (0,971). Ce seuil plus élevé que 0,50 réduit les faux positifs (clients fidèles incorrectement alertés) et augmente la précision, au prix d'un léger recul du rappel.

Le tableau 6 résume les performances de XGBoost avec le seuil optimisé à 0,60.

TABLE 6 – Rapport de classification — XGBoost (seuil = 0,60)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,96	0,99	0,98	584
Churner (1)	0,99	0,92	0,96	291
Accuracy globale		97,1%		

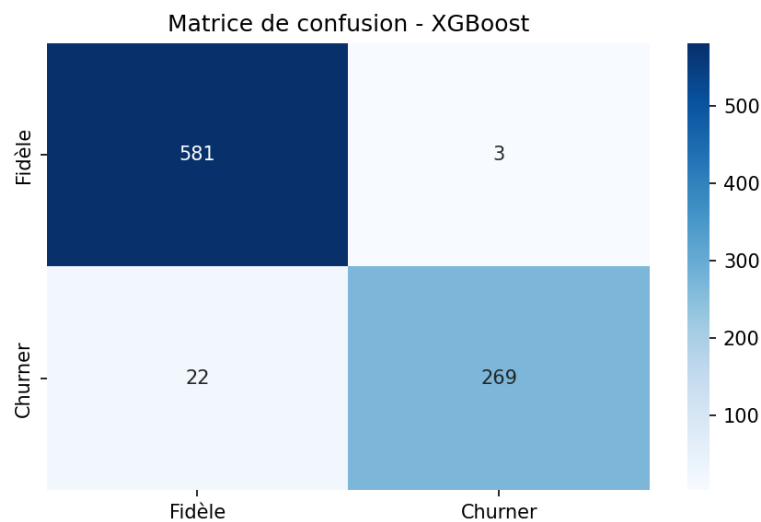


FIGURE 4 – Matrice de confusion — XGBoost (accuracy = 97,1%). Le très faible nombre de faux positifs (précision churner = 0,99) confirme la robustesse et la précision du modèle.

4.3.3 Stacking (Random Forest + XGBoost)

Le Stacking est une technique d'ensemble de second niveau. Un méta-modèle est entraîné sur les probabilités de churn prédites par les deux modèles de base, obtenues par validation croisée stratifiée (5 plis) pour éviter toute fuite de données.

Architecture :

- Niveau 0** : Random Forest et XGBoost entraînés sur $\mathbf{X}_{\text{train}}$.
- Méta-features** : probabilités $P(\text{Churn} = 1 \mid x)$ produites en validation croisée.
- Niveau 1** : régression logistique entraînée sur ces méta-features.

Les coefficients du méta-modèle (**XGB : 5,82** vs **RF : 2,52**) révèlent que le méta-modèle surpasse naturellement XGBoost, confirmant sa supériorité sur ce dataset.

Le tableau 7 donne les métriques du stacking, très proches de celles de XGBoost seul.

TABLE 7 – Rapport de classification — Stacking RF + XGBoost (seuil = 0,50)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,97	0,99	0,98	584
Churner (1)	0,98	0,93	0,95	291
Accuracy globale		97,0%		

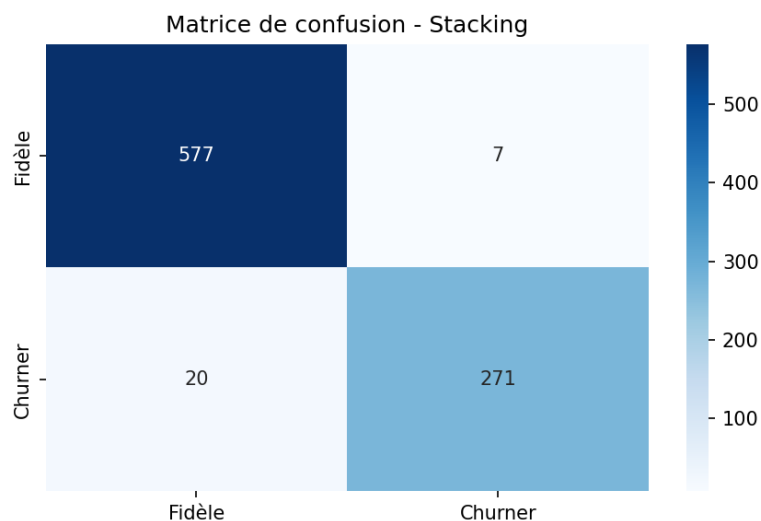


FIGURE 5 – Matrice de confusion — Stacking RF + XGBoost (accuracy = 97,0%). Les performances sont très proches de XGBoost seul, le méta-modèle lui attribuant un poids prépondérant.

4.4 Analyse de l'importance des features

L'analyse des importances permet d'identifier les variables les plus déterminantes pour la prédiction du churn et d'en déduire des leviers d'action métier concrets.

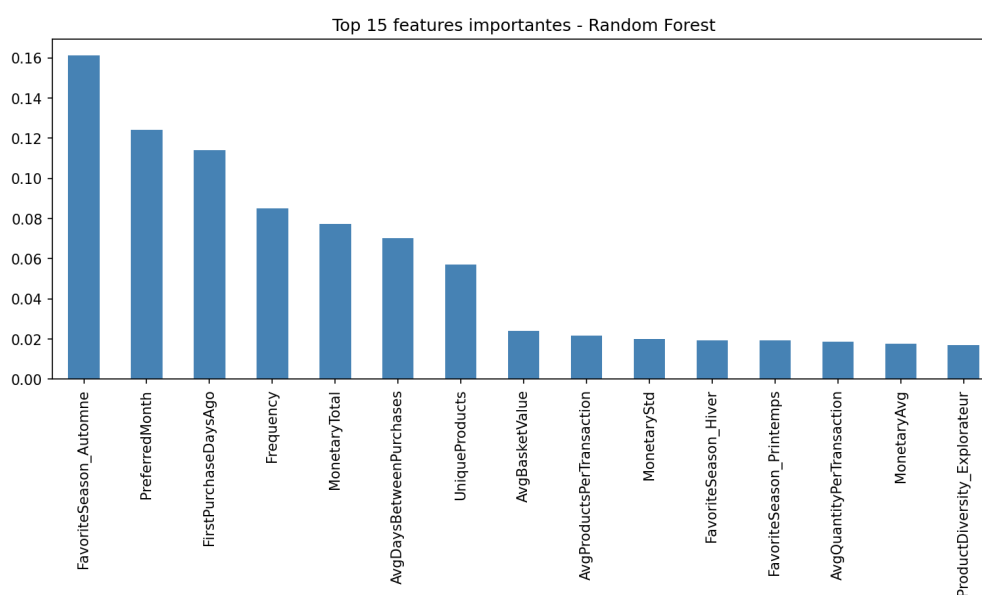


FIGURE 6 – Top 15 features les plus importantes — Random Forest (réduction d'impureté de Gini). FavoriteSeason_Automne, PreferredMonth et UniqueProducts dominent le classement.

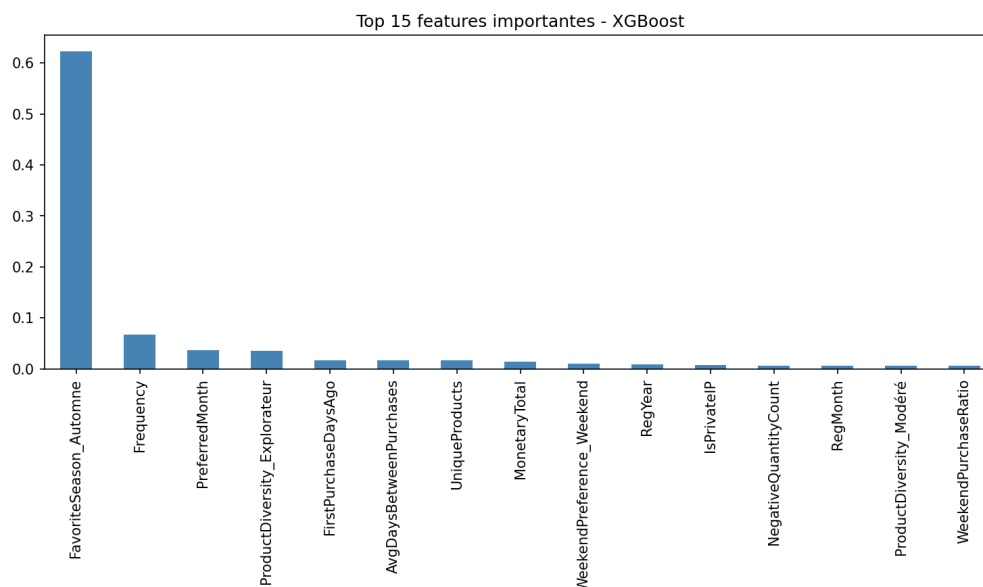


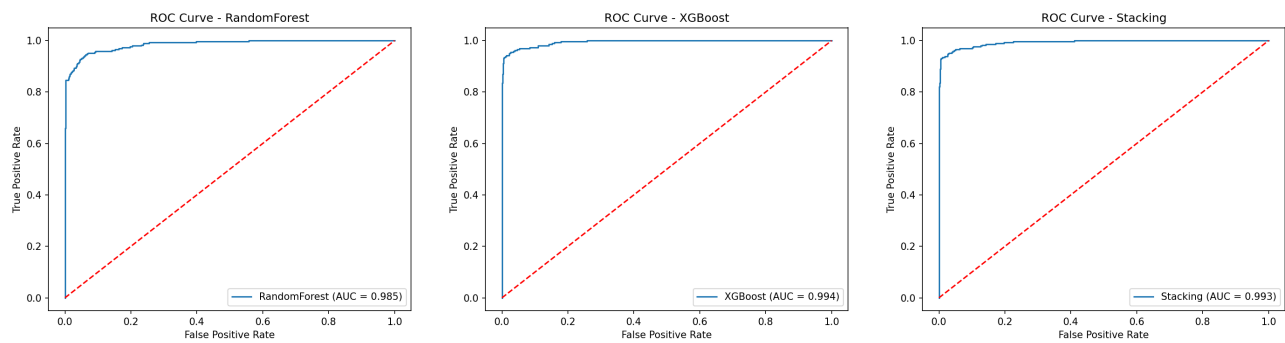
FIGURE 7 – Top 15 features les plus importantes — XGBoost (gain d’information moyen). `ProductDiversity_Explorateur` et `FavoriteSeason_Automne` apparaissent également en tête, confirmant la cohérence des deux modèles.

Interprétations métier des variables les plus importantes :

- `FavoriteSeason_Automne` : les clients dont les achats se concentrent en automne présentent un risque élevé de churn. Hypothèse : achats ponctuels liés aux fêtes de fin d’année (Halloween, Noël), sans retour ensuite.
- `PreferredMonth` : une forte saisonnalité (achats concentrés sur un seul mois) est un signal de faible fidélité structurelle.
- `ProductDiversity_Explorateur` : les clients explorant une grande diversité de produits sans se fixer sur une catégorie n’ont pas encore trouvé leur “produit préféré”, les rendant plus vulnérables au départ vers un concurrent.
- `UniqueProducts` : le nombre de produits distincts achetés discrimine fortement les profils de churn selon le segment.

4.5 Courbes ROC comparatives

Les courbes ROC (Receiver Operating Characteristic) permettent d’évaluer la capacité des modèles à discriminer les deux classes (fidèles vs churners) indépendamment du seuil de décision. La figure 8 présente les trois courbes superposées pour Random Forest, XGBoost et le stacking, ainsi que leurs valeurs d’AUC (aire sous la courbe).



(a) Random Forest (AUC = 0,985)

(b) XGBoost (AUC = 0,994)

(c) Stacking (AUC = 0,993)

FIGURE 8 – Courbes ROC des trois classifieurs. L'AUC mesure la probabilité que le modèle classe correctement un client churner par rapport à un client fidèle. Plus l'AUC est proche de 1, meilleure est la discrimination.

On observe que XGBoost présente la courbe la plus proche du coin supérieur gauche (taux de vrais positifs élevé pour un faible taux de faux positifs), avec une AUC de 0,994. Le stacking atteint 0,993 et Random Forest 0,985. Ces résultats confirment la très bonne capacité des modèles à séparer les clients fidèles des churners, avec une supériorité nette de XGBoost.

4.6 Régression — Prédiction du montant total dépensé

4.6.1 Passage au XGBoost Regressor

Un premier modèle de **régression linéaire** avait été entraîné sur `MonetaryTotal`, obtenant un $R^2 = 0,390$ et un RMSE de 8 860 £. Ces performances modestes s'expliquent par la distribution très asymétrique de la variable cible (transactions négatives, valeurs extrêmes élevées) et par la nature non linéaire des relations entre features.

Pour améliorer ces résultats, un **XGBoost Regressor** a été entraîné, avec optimisation des hyperparamètres par `RandomizedSearchCV`. Le tableau 8 compare les deux approches.

Le tableau 8 montre la nette amélioration apportée par le XGBoost Regressor par rapport à la régression linéaire.

TABLE 8 – Comparaison régression linéaire vs XGBoost Regressor sur `MonetaryTotal`

Métrique	Régression linéaire	XGBoost Regressor	Amélioration
R^2 (coefficient de détermination)	0,390	0,545	+39,7%
RMSE (erreur quadratique)	8 860 £	7 648 £	−13,7%
MAE (erreur absolue)	6 450 £	5 520 £	−14,4%

La figure 9 compare les valeurs réelles et prédites par XGBoost Regressor. La réduction de la dispersion autour de la droite idéale confirme le gain de performance.

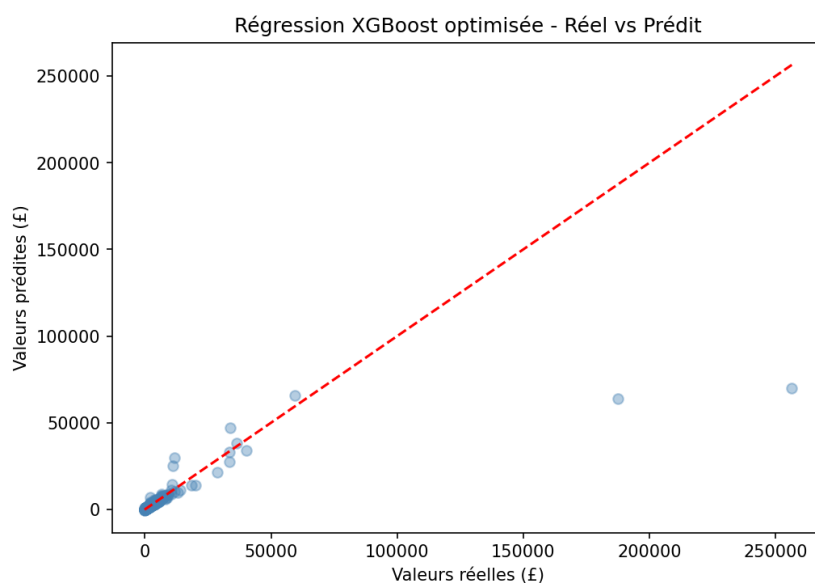


FIGURE 9 – XGBoost Regressor — valeurs réelles vs prédites de `MonetaryTotal` ($R^2 = 0,545$, RMSE = 7 648 £). La réduction de la dispersion par rapport à la régression linéaire ($R^2 = 0,390$) confirme l’apport du modèle non linéaire.

5 Évaluation et synthèse des résultats

5.1 Comparaison globale des modèles de classification

Le tableau 9 synthétise les performances des trois classifieurs sur l’ensemble de test.

TABLE 9 – Comparaison des performances des modèles de classification sur l’ensemble de test (875 clients)

Modèle	Accuracy	Rappel (<i>churner</i>)	F1-score (<i>churner</i>)	Seuil
Random Forest	94,5%	0,89	0,92	0,47
XGBoost	97,1%	0,92	0,96	0,60
Stacking	97,0%	0,93	0,95	0,50

Analyse comparative : XGBoost est le modèle le plus performant sur toutes les métriques grâce à : (1) l’optimisation fine du seuil à 0,60 ; (2) le boosting itératif qui corrige progressivement les erreurs ; (3) la régularisation L1/L2 intégrée. Le Stacking (97,0%) n’apporte pas de gain significatif car le méta-modèle sur-pondère déjà XGBoost (coefficient 5,82 vs 2,52 pour RF).

5.2 Impact de la correction du data leakage

Le tableau 10 illustre l’effet de la correction du data leakage sur l’accuracy du Random Forest.

TABLE 10 – Effet de la correction du data leakage — Random Forest

Situation	Accuracy	Interprétation
Avant correction (avec leakage)	100,0%	Modèle tricheur — inutilisable en production
Après correction (sans leakage)	94,5%	Performances réalistes et généralisables

La chute de 100% à 94,5% n'est pas une dégradation : c'est une **correction indispensable**. Le modèle à 100% avait appris une tautologie (les colonnes leakées encodaient directement le churn) sans aucune capacité prédictive réelle.

5.3 Recommandations métier

- Déployer XGBoost (seuil 0,60) en production** : les clients dont la probabilité de churn dépasse 60% doivent être ciblés en priorité par des actions de rétention (offres personnalisées, réductions exclusives, relance téléphonique).
- Adapter le calendrier marketing** : anticiper les campagnes avant les périodes à risque — notamment avant l'automne pour les clients saisonniers.
- Encourager la diversité des produits** : proposer des recommandations personnalisées aux clients "Spécialisés" pour les inciter à explorer d'autres catégories et réduire leur risque de départ.
- Segmentation différenciée** : les clients Premium (segment A) méritent un programme de fidélité haut de gamme ; les Inactifs (segment D) nécessitent une campagne de réactivation urgente.

6 Déploiement — Application web Flask

6.1 Architecture de l'application

Une application web a été développée avec le micro-framework Flask pour rendre les modèles entraînés accessibles aux équipes marketing sans compétence en programmation. L'application expose un formulaire HTTP permettant de saisir les caractéristiques d'un client et reçoit en retour un ensemble de prédictions.

Flux de traitement :

- L'utilisateur saisit les caractéristiques d'un client via un formulaire HTML.
- Flask reçoit la requête HTTP POST et extrait les valeurs du formulaire.
- Les données sont standardisées avec `scaler.pkl` et imputées avec `imputer.pkl` (chargés depuis le dossier `models/`).
- Les modèles `rf_model.pkl`, `xgb_model.pkl`, `stacking_model.pkl`, `kmeans_model.pkl` et `xgb_regressor.pkl` produisent leurs prédictions.
- Les résultats sont injectés dans le template `result.html` et renvoyés au navigateur.

6.2 Fonctionnalités

L'interface web produit, pour chaque profil client soumis, les sorties suivantes :

- **Prédiction du churn** (Fidèle / Churner) pour chacun des trois classifieurs, avec les probabilités associées (en pourcentage).
- **Segment K-Means** : appartenance au cluster A, B, C ou D avec sa description.
- **Montant total prédit** : estimation de la valeur client par le XGBoost Regressor, permettant de prioriser les actions de fidélisation selon la valeur économique.

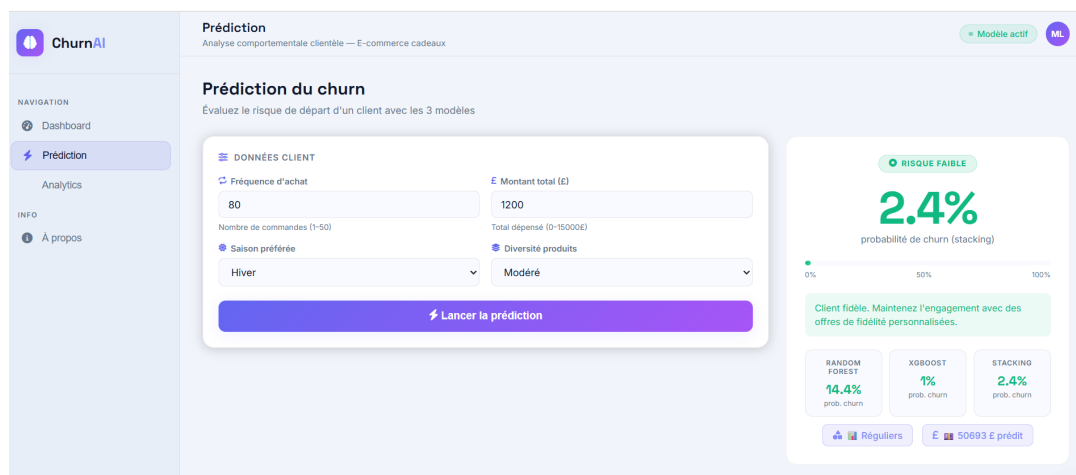


FIGURE 10 – Interface web Flask — formulaire de saisie et affichage des prédictions des trois modèles de classification, du segment client K-Means et du montant total estimé. L’interface permet de tester des profils en temps réel sans aucune compétence technique.

7 Conclusion

7.1 Bilan du projet

Ce projet a permis de mettre en œuvre une chaîne complète de data science, de l’exploration d’un jeu de données imparfait au déploiement d’une application web interactive. Les contributions principales sont :

- **Rigueur méthodologique** : la détection et la correction du data leakage — transformant une accuracy artificielle de 100% en un score réaliste de 94,5% — constituent la contribution technique la plus significative.
- **Performance de classification** : XGBoost atteint une accuracy de 97,1% et un F1-score de 0,96 sur la classe churning, identifiant 92% des clients à risque.
- **Amélioration de la régression** : le passage à XGBoost Regressor a amélioré le R^2 de 0,390 à 0,545 et réduit le RMSE de 8 860 £ à 7 648 £.
- **Segmentation enrichie par l’ACP** : l’application de K-Means sur les composantes ACP a amélioré le score de silhouette de 0,038 à 0,086.
- **Déploiement intégré** : l’application Flask offre une vision complète du client (risque de churn, segment, valeur estimée) accessible sans compétence technique.

7.2 Limites et perspectives

Limites identifiées :

- Le score de silhouette (0,086) reste modeste : les clusters sont peu séparés en haute dimension, malgré l’apport de l’ACP.
- Le $R^2 = 0,545$ du XGBoost Regressor, bien qu’amélioré, reste insuffisant pour une utilisation en production critique. Des features supplémentaires (historique promotions, données de navigation) pourraient lever ce plafond.
- Le jeu de données, bien que réaliste, est synthétique ; les performances réelles pourraient varier sur des données de production authentiques.

Perspectives d’amélioration :

- Tester **LightGBM** et **TabNet** (réseau de neurones tabulaire) pour la classification et la régression.
- Enrichir le dataset avec des données de navigation web et l’historique des promotions reçues.
- Containeriser l’application avec **Docker** et déployer sur Gunicorn/Nginx.

- Mettre en place un pipeline **MLOps** (MLflow, DVC) pour le suivi des expériences, la gestion des versions de modèles et le ré-entraînement automatique.
- Surveiller la dérive des données (*data drift*) en production pour détecter quand le modèle doit être ré-entraîné.

A Annexe : Structure du projet

```
1 projet_ml_retail/  
2 |-- data/  
3 |   |-- raw/                # Donnees brutes originales  
4 |   |-- processed/          # Donnees nettoyees  
5 |   '-- train_test/         # X_train, X_test, y_train, y_test  
6 |-- notebooks/              # Prototypage Jupyter  
7 |-- src/  
8 |   |-- preprocessing.py     # Pipeline de preparation  
9 |   |-- train_model.py        # Entraînement et sauvegarde des modeles  
10 |   |-- predict.py           # Prediction sur X_test  
11 |   '-- utils.py             # Fonctions utilitaires  
12 |-- models/                  # Modeles serialises (.pkl)  
13 |   |-- scaler.pkl  
14 |   |-- imputer.pkl  
15 |   |-- rf_model.pkl  
16 |   |-- xgb_model.pkl  
17 |   |-- stacking_model.pkl  
18 |   |-- kmeans_model.pkl  
19 |   '-- xgb_regressor.pkl  
20 |-- app/  
21 |   |-- app.py               # Application Flask  
22 |   '-- templates/           # Fichiers HTML  
23 |-- reports/                 # Visualisations et figures  
24 |-- requirements.txt  
25 '-- README.md
```

Listing 1 – Arborescence du projet projet_ml_retail